MONITOR Z · L00.2

Clase 2 — La misma idea en z/OS: STC, JES y address spaces

Cuaderno de estudio · versión del 2026-10-26

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

L00.2 — Clase 2 · La misma idea en z/OS: STC, JES y address spaces

Temario de la L00: 1. El demonio en Unix ✅ · **2. La misma idea en z/OS** · 3. Por qué no alcanza con «preguntar cada tanto» · 4. Las piezas del monitor, y dónde estamos hoy.

Objetivo de esta clase: saber **dónde vive** ZTKMON (un address space), **quién lo hace correr** (JES2 y un iniciador) y **qué cambia** cuando pase a STC. Todo leído en el spool real del monitor.

> 📚 Fuentes: - *z/OS MVS JCL Reference* y *z/OS JES2 Introduction*: job, clase, iniciador, spool. - *z/OS MVS System Messages* (`IEF` y `$HASP`): `$HASP373`, `$HASP395`, `IEF032I`, `IEF142I`. - *z/OS MVS Programming: Assembler Services Guide*: address space, tareas (TCB), `EXTRACT` / `QEDIT` para STC. - El spool de **JOB34879** (ZTKMONR, DESA, 2026-10-22): `JESJCL`, `JESMSGLG` y `JESYSMSG`. - **JOB25949** (ZTKMONR, 2026-10-08): `+DIAG FASE …` `Q=00013`. **JOB26889** (sonda ZTKMETA, 2026-10-09): `+TOTAL` y `+MET … AS=`. - *L07 - Ser un STC de verdad*, sección 1 (la tabla job contra STC).

1. El address space: el departamento de cada programa

Analogía: z/OS es un **edificio de departamentos**. Cada programa que corre vive en su departamento, con su propia numeración de habitaciones: la «habitación 1000» de un departamento no es la del vecino. Ese departamento es el **address space**, y su número es la **ASID**. En Unix es el **proceso**, con su memoria virtual propia.

Dato	Valor	Testigo
Address spaces en uso en DESA	**175**	`+TOTAL V=00001 AS=00175 JOBS=00036`, JOB26889 (09-10)
De ellos, jobs batch de `P*`	**36**, con ASID entre `X'003B'` y `X'0138'`	las líneas `+MET … AS=003B …` de JOB26889
El resto (139)	el sistema, JES2, TCP/IP, CICS, los STC…	**No se miró uno por uno**

Lo que usó ZTKMON debajo de la línea	**92K**	`IEF032I … VIRT: 92K` de JOB34879
Lo que usó encima de la línea	**1.124K**: casi todo es el anillo (1.114.112 bytes = 1.088K)	`EXT: 1124K` y `+DIAG ANILLO … EN=19B00000`

Adentro del departamento viven las tareas (TCB), que son el equivalente de los *threads*. ZTKMON tiene **una sola tarea propia**: en el fuente no hay ningún `ATTACH`. Las tareas que el sistema pone en cada address space existen, pero **no las vemos** desde el listado.



Lámina L00-13_address_space

2. JES2 y el iniciador: quién hace correr un job

Analogía: JES2 es la **recepción** del edificio. Recibe el trabajo, lo anota, lo hace esperar en la sala y, cuando se libera un **mostrador** (el iniciador) de su clase, lo hace pasar. Todo lo que el trabajo «dice» queda en el **libro de salida** (el spool).

Así corrió ZTKMON el 22-10, mensaje por mensaje:

Hora	Mensaje (JESMSGLG / JESYSMSG)	Qué pasó
11.37.14	`IRR010I USERID PSJB IS ASSIGNED TO THIS JOB`	RACF le pone **tu** usuario

11.37.14	`$HASP373 ZTKMONR STARTED - INIT 11 - CLASS P - SYS DESA`	el **iniciador 11**, de clase P, lo toma
11.37.14	`IEF236I ALLOC. FOR ZTKMONR MONITOR`	reserva `STEPLIB` (volumen `O4DWKB`) y `SYSPRINT` (al spool)
…	30 minutos de vueltas	el iniciador 11 queda **ocupado** todo ese tiempo
12.07.16	`IEF142I ZTKMONR MONITOR - STEP WAS EXECUTED - COND CODE 0000`	termina el paso
12.07.16	`$HASP395 ZTKMONR ENDED - RC=0000`	el iniciador 11 queda libre

El precio del job batch: `CLOCK 30.02` minutos con `CPU .04`. El iniciador estuvo **ocupado 30 minutos** para un programa que trabajó **2,55 segundos**. Es como dejar a un cliente sentado en el mostrador toda la noche aunque casi no hable. Cuántos iniciadores de clase P tiene DESA **no está medido**: si son pocos, el monitor le quita uno a los demás jobs `P`.

JES2 encola por nombre repetido

JES2 no deja correr **dos jobs con el mismo nombre a la vez** (en esta instalación y para estos jobs: es lo que se vio, no una regla probada para todas las clases). El 08-10, en JOB25949:

```
+EVT INI  ZTKDOK   JOB25951 OW=PSJB CL=P T=2026-10-08-10:58:41   ← uno e
+DIAG FASE ZTKDOK  JOB25952 Q=00013                               ← y los
+DIAG FASE ZTKDOK  JOB25953 Q=00013                                  en l
 …           JOB25954, JOB25955, JOB25956  Q=00013
```

Por eso el JCL de ZTKMONVR avisa: *«NO PUEDE CORRER JUNTO A OTRO ZTKMONR (JES2 LOS ENCOLA)»*. Y el nombre no se puede cambiar: Sistemas reservó el **puerto 8010 para el job** `ZTKMONR` (2026-10-08, cabecera de ZTKMON.asm).

JES2 y el iniciador: así corrió ZTKMON el 22-10

L00 · Clase 2 · STC, JES y address spaces

Hora	Mensaje	Qué pasó
11.37.14	`IRR010I USERID PSJB IS ASSIGNED TO THIS JOB`	🔑 RACF le pone **tu** usuario
11.37.14	`$HASP373 ZTKMONR STARTED - INIT 11 - CLASS P`	🪑 el **iniciador 11** lo toma
11.37.14	`IEF236I ALLOC. FOR ZTKMONR MONITOR`	📦 STEPLIB (O4DWKB) y SYSPRINT al spool
12.07.16	`IEF142I … STEP WAS EXECUTED - COND CODE 0000`	✅ termina el paso
12.07.16	`$HASP395 ZTKMONR ENDED - RC=0000`	🪑 el iniciador 11 queda libre

🛎️ Analogía: la recepción del edificio

JES2 recibe el trabajo, lo anota y lo hace esperar en la sala. Cuando se libera un **mostrador** de su clase (el iniciador), lo hace pasar. Lo que dice queda en el libro de salida: el **spool**.

💸 El precio del job batch

`CLOCK 30.02` con `CPU .04`: el INIT 11 estuvo **ocupado 30 minutos** por **2,55 s** de trabajo. Cuántos iniciadores de clase P tiene DESA: **no medido**.

Fuente: JESMSGLG y JESYSMSG de JOB34879 (DESA, 2026-10-22)

Lámina L00-14_jes_e_iniciador

JES2 no deja correr dos jobs con el mismo nombre

L00 · Clase 2 · STC, JES y address spaces

```
JOB25949 · ZTKMONR · 2026-10-08
+EVT INI  ZTKDOK   JOB25951 OW=PSJB CL=P T=2026-10-08-10:58:41   ← uno ejecuta
+DIAG FASE ZTKDOK  JOB25955 Q=00013
+DIAG FASE ZTKDOK  JOB25954 Q=00013                              ← los otros cinco esperan
+DIAG FASE ZTKDOK  JOB25953 Q=00013                                 en la fase 13 (STAT_SELECT)
+DIAG FASE ZTKDOK  JOB25952 Q=00013
+DIAG FASE ZTKDOK  JOB25956 Q=00013
```

📜 Por eso el JCL avisa

«NO PUEDE CORRER JUNTO A OTRO ZTKMONR (JES2 LOS ENCOLA)» (JCL ZTKMONVR).

🔒 Y el nombre no se cambia

Sistemas reservó el **puerto 8010 para el job ZTKMONR** (2026-10-08, cabecera de ZTKMON.asm).

⚠️ Alcance

Es lo que **se vio** en esta instalación con estos jobs. No se probó para todas las clases.

Fuente: SYSPRINT de JOB25949 (líneas 16-21) · JCL ZTKMONVR · ZTKMON.asm v0.9 (cabecera)

Lámina L00-15_nombre_repetido

3. El STC: el demonio de verdad

Un **STC** (*started task*) es un programa que arranca el **operador** o el arranque del sistema, no un `SUBMIT`. Es el equivalente directo del demonio de Unix.

	📄 Job batch (ZTKMONR hoy)	🏭 STC (ZTKMON en la L07)	🐧 Unix
Arranca con	`SUBMIT` (lo hacés vos)	`S ZTKMON` (operador o arranque)	`systemctl start` / el arranque

El JCL sale de	el miembro que sometés	una **PROC** en la **PROCLIB**	el archivo de la unidad
Iniciador	**ocupa uno**: INIT 11 durante 30,02 min	**no usa iniciador batch**: tiene su address space propio	—
Usuario	el tuyo: `PSJB` (`IRR010I`)	el que asigna RACF por la clase **STARTED**	el `User=` de la unidad
Se para con	que termine solo (30 min por PARM) o cancelándolo	`P ZTKMON`, ordenado	`SIGTERM`
Se le habla con	—	`F ZTKMON,texto`	`SIGHUP`, o un socket
Lo pide a Sistemas	el puerto (hecho)	la PROC en la PROCLIB y el perfil STARTED	—

El programa casi no cambia. Cambian cómo se arranca, que deja de ocupar un iniciador, y que tiene que **escuchar al operador** (`EXTRACT … FIELDS=COMM`, el CIB y `QEDIT`, en la L07). Nada de esto está hecho todavía.

Job batch contra STC: el demonio de verdad

L00 · Clase 2 · STC, JES y address spaces

	Job batch (ZTKMONR hoy)	STC (ZTKMON en la L07)	Unix
Arranca con	`SUBMIT`: vos	`S ZTKMON`: operador o arranque	`systemctl start`
El JCL sale de	el miembro que sometés	una **PROC** en la **PROCLIB**	el archivo de la unidad
Iniciador	**ocupa uno: INIT 11, 30,02 min**	**ninguno: address space propio**	—
Usuario	`PSJB` (IRR010I)	el de RACF por la clase **STARTED**	`User=`
Se para con	reloj (30 min) o cancelándolo	`P ZTKMON`, ordenado	`SIGTERM`
Se le habla con	—	`F ZTKMON,texto`	`SIGHUP` / un socket
Pide a Sistemas	el puerto (hecho)	la PROC y el perfil STARTED	—

> **El programa casi no cambia.** Cambia cómo arranca, que deja de ocupar un iniciador y que tiene que escuchar al operador (`EXTRACT … FIELDS=COMM`, CIB, `QEDIT`). **Nada de esto está hecho todavía**: es la L07.

Fuente: L07, sección 1 · JESMSGLG y JESYSMSG de JOB34879 · z/OS MVS Programming: Assembler Services Guide (EXTRACT, QEDIT)

Lámina L00-16_job_contra_stc

4. La tabla completa: de Unix a z/OS

Unix	z/OS	En nuestro proyecto

proceso	**address space** (ASID)	uno de los 175 de DESA
thread	**tarea (TCB)**	ZTKMON tiene una sola tarea propia
`fork` + `exec` desde la shell	`SUBMIT` → JES2 → iniciador	`$HASP373 … INIT 11`
demonio de `systemd`	**STC** con su PROC	L07
`sleep`, semáforo, `select`	`STIMER`, ECB (`WAIT` / `POST`), `SELECT` de EZASOKET	L00.1, L02, L06
`SIGTERM` / `SIGHUP`	`P` / `F`	L07
`syslog`, stdout	`WTO` a la consola, `SYSPRINT` al spool	`+HOLA`, `+EVT`, `+DIAG`
`ps`	preguntarle a JES2 por **SSI 80**	L03, y la clase 3
root	**APF** + estado supervisor	sólo lo pide ENF 70 (L08)

5. Qué cambia para el monitor (lo que viene)

Hoy	Mañana	Qué lo trae
job ZTKMONR que ocupa el INIT 11	STC sin iniciador	L07 + Sistemas (PROCLIB, STARTED)
termina por reloj a los 30 min	vive hasta `P ZTKMON`	L07
corre con `PSJB`	corre con el usuario del STC	RACF, clase STARTED
pregunta a JES cada 5 s	además, JES le avisa (ENF 70)	L08 + APF

Resumen

Idea	En una línea	Testigo
Address space	el departamento de cada programa, con su ASID	175 en DESA (JOB26889)

ZTKMON adentro	92K debajo de la línea y 1.124K encima (el anillo)	`IEF032I` de JOB34879
JES2	recibe, encola, asigna iniciador y guarda la salida en el spool	`$HASP373` / `$HASP395`
El iniciador	el mostrador: un job lo ocupa todo el tiempo que corre	INIT 11, 30,02 min por 2,55 s de CPU
Nombre repetido	JES2 encoló cinco ZTKDOK con fase 13	JOB25949
STC	el demonio de verdad: `S`, `P`, `F`, sin iniciador	L07 (no hecho)

Preguntas de comprobación

1. JOB34879 tuvo ocupado el **INIT 11** 30,02 minutos y gastó 2,55 s de CPU. Si ZTKMON corriera toda la noche del cierre

como job de clase P, ¿a quién perjudica y por qué? ¿Cómo lo resuelve el STC?

1. En JOB25949, cinco ZTKDOK esperaban con `Q=00013`. ¿Por qué esperaban? ¿Y qué pasaría si sometés un segundo ZTKMONR

mientras el primero corre?

Mis respuestas

Relacionado: *L00.1 - Clase 1 - El demonio en Unix - la idea que ya conoces* · *L07 - Ser un STC de verdad* · *L03 - Hablarle a JES por SSI 80* · siguiente: L00.3.

Notas